COMP90077
Lecture 4
Knapsack and Set Cover

William Umboh

University of Melbourne

Caution

- Slides are designed as presentation aid to go along with lecture, not as study material
- Slides not designed to make sense on their own

Assignment 1

- Take a look ASAP if you have not already
- Problems usually require subconscious processing
- Spend half an hour to load the problems so that your subconscious can work on it in the background
- Any partners?
- Feel free to approach me over email/consultation if you are not sure of level of details required in responses (proofs, reflection, etc)

Feedback from Survey

- Link to survey
- Closes tomorrow (27/3/2026)
- 14/16 respondents from Tuesday tutorial
- Level of understanding

	Lecture	Tutorial
Week 1	4	3.8
Week 2	3.4	3.3
Week 3	2.9	2.9

  - Lecture 2 > Lecture 3: surprising since I used more pictures and intuition for proofs and audience seemed to follow better
  - Maybe students have more time to digest material from earlier weeks?
  - Normal to leave proof-based lectures with 3/5

Unit Website

- Happy with response: 1 Ineffective, 3 neutral, 5 effective, 5 very effective
- Rationale for MystMD website vs LMS:
  - Low friction for you
    - Tooltip popup and links for definitions and theorems
    - Mobile-friendly
    - Global search across notes, tutorials, tutorial solutions
    - Downloadable markdown source for your own notes
  - Low friction for me (commit + git push vs compile + okta + navigate to modules + upload PDF)

Slides and Notes

- Handwriting proofs
  - More natural, and helps ensure I don’t go too fast
  - Take notes as I go through the proof
  - Much easier to draw diagrams
- Level of detail
  - 1 comment asking for “a lot more detail in study notes”
  - Not sure what kind of details is needed? Tell me or student reps

During Lectures

- Grateful to you for attending 2-hr in-person lectures
- Feel free to stop me with questions:
  - Why is this interesting?
  - How does this relate to the big picture?
  - Can you help me visualize what's going on?
  - What's the intuition behind this proof/calculation?
  - I blinked, can you repeat that again?
  - Can you remind me what the definition of XXX is?
  - Can we take a break?

Today

- Last lecture on greedy
- New type of greedy algorithm
- Applications: Knapsack and Set Cover Problems

Recap

- Problems considered thus far of the form

$$\max_{S \in \mathcal{F}} g(S)$$

where feasible sets are downwards-closed (i.e. independence system)

- Week 2: When g is additive, greedy is optimal if and only if feasible sets form a matroid
- Week 3: When g is monotone submodular, greedy is a good approximation if feasible sets are sets of size at most k
- Do you remember the description of these algorithms?

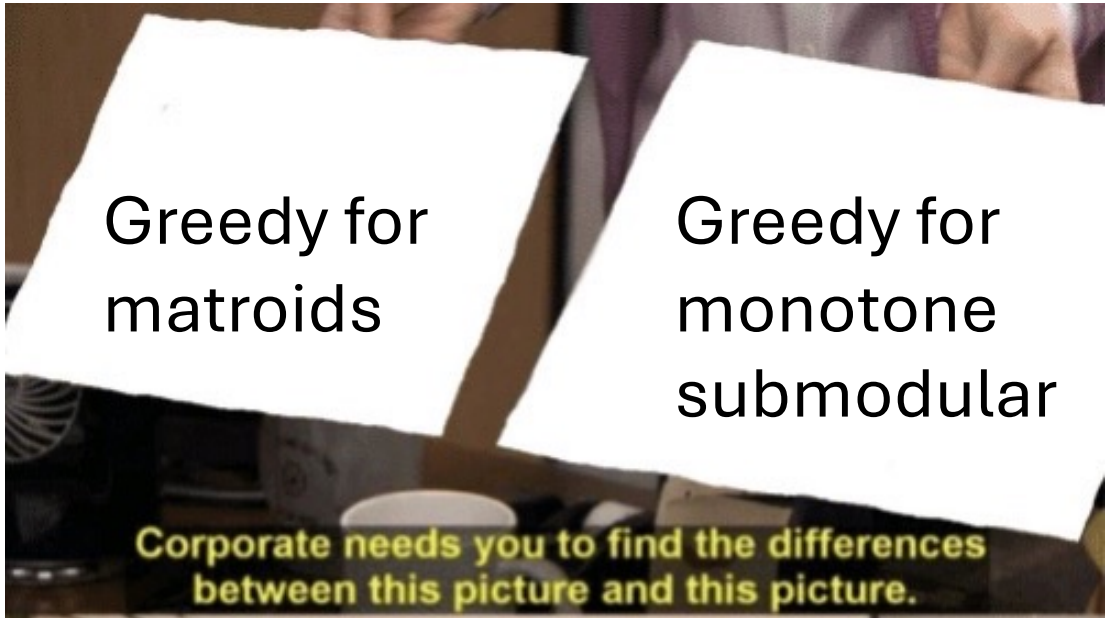
Greedy Algorithms

Matroids

- Initialize $B \leftarrow \emptyset$
- For each $e \in E$ (in decreasing order of weight):
  - Add e to B if $B \cup \{e\}$ independent
- Return B

Monotone Submodular Maximization

- Initialize $I \leftarrow \emptyset$
- While $|I| < k$:
  - Find j^* that maximizes increase in objective: $g(I \cup \{j^*\}) - g(I)$
  - Add j^* to I
- Return I


Greedy for matroids
Greedy for monotone submodular
Corporate needs you to find the differences between this picture and this picture.


Greedy for matroids
Greedy for monotone submodular
Corporate needs you to find the differences between this picture and this picture.
They're the same picture.
imgflip.com

Marginal Greedy for Independence Systems

$$\max_{S \in \mathcal{F}} g(S)$$

- Initialize $I \leftarrow \emptyset$
- While I not maximally feasible:
  - Find e that maximizes increase in objective function: $g(I \cup \{e\}) - g(I)$
  - Add e to I
- Return I

Both greedy for matroids and greedy for monotone submodular have this form

Knapsack

- Given items with weights and sizes, and a knapsack of capacity C find max-weight subset with total size at most C
- Example: What to see at Melbourne International Comedy Festival?
  - C = budget
  - Items = shows, weight = your preference for show, size = ticket price

C = 70

1 2 3

Image Source: comedyfestival.com.au

Weight: 5
Price: 30

Weight: 4
Price: 30

Weight: 3
Price: 30

- Is Marginal Greedy any good?

Is Marginal Greedy any good?

- Given items with weight and sizes, and a knapsack of capacity C find max-weight subset with total size at most C
- Example: What to see at Melbourne International Comedy Festival?
  - C = budget
  - Items = shows, weight = your preference for show, size = ticket price

C = 70

1 2 3

Image Source: comedyfestival.com.au

Weight: 5
Price: 30

Weight: 4
Price: 30

Weight: 3
Price: 30

- On this instance, Marginal Greedy picks {1, 2} which is optimal

Is Marginal Greedy any good?

- Given items with weight and sizes, and a knapsack of capacity C find max-weight subset with total size at most C
- Example: What to see at Melbourne International Comedy Festival?
  - C = budget
  - Items = shows, weight = your preference for show, size = ticket price

C = 70

1 2 3

Image Source: comedyfestival.com.au

1	2	3
Weight: 5 Price: 70	Weight: 4 Price: 30	Weight: 3 Price: 30

- On this instance, Marginal Greedy picks {1} which is sub-optimal

Is Marginal Greedy any good?

- Given items with weight and sizes, and a knapsack of capacity C find max-weight subset with total size at most C
- Example: What to see at Melbourne International Comedy Festival?
  - C = budget
  - Items = shows, weight = your preference for show, size = ticket price

1 2 3 ..... n

C = n

Weight: 2
Price: n

Weight: 1
Price: 1

Weight: 1
Price: 1

- On this instance, Marginal Greedy picks {1} which is sub-optimal

Approximation Ratio Tiers

- S: 1

- A: Constant $\Omega(1)$ for maximization, $O(1)$ for minimization
- B: Logarithmic $\Omega(1/\log n)$ for maximization, $O(\log n)$ for minimization

- F: Linear $\Omega(1/n)$ for maximization, $O(n)$ for minimization

How to get a better greedy algorithm?

- Let's take a look at the bad instance again to get some intuition
- Is there a better ordering of items?
- Intuitively, the optimal solution picks elements with better bang-per-buck, i.e. weight/price

C = n

1 2 3 ..... n

Weight: 2
Price: n

Weight: 1
Price: 1

Weight: 1
Price: 1

Knapsack – Density Greedy

- Given items $e_1, \ldots, e_n$ with weights $w(e_i)$ and sizes $s(e_i)$, and a knapsack of capacity C find

  subset A that maximizes $w(A)$ such that $s(A) \leq C$

- Def. The density of item e_i is $w(e_i)/s(e_i)$

Density Greedy Algorithm

- Initialize $A \leftarrow \emptyset$
- For each item e_i in decreasing order of density:
  - If $w(e_i) + w(A) \leq C$, add e_i to A
  - Else, break
- Return A

Does this work? 🧐

Approximation ratio?

Returns the largest feasible prefix of items sorted in decreasing order of density

Knapsack – Density Greedy Bad Instance

- Initialize $A \leftarrow \emptyset$
- For each item e_i in decreasing order of density $w(e_i)/s(e_i)$:
  - If $w(e_i) + w(A) \leq C$, add e_i to A
  - Else, break
- Return A

C =

1 2 3

Weight:
Price:

Weight:
Price:

Weight:
Price:

Best of {Density Greedy, Single Item}

Knapsack Algorithm

- Initialize $A \leftarrow \emptyset$
- For each item e_i in decreasing order of density:
  - If $w(e_i) + w(A) \leq C$, add e_i to A
  - Else, break
- Let e^* be item with largest weight
- If $w(e^*) \geq w(A)$
  - Return single item e^*
- Else
  - Return A

Knapsack Algorithm is (1/2)-Approximation

Recall Max Coverage

- Given a bound k and a set system
  - A *universe* U of n elements
  - A collection of m sets $S_1, \dots, S_m$
- Goal: find k sets with maximum coverage
- Equivalently, find $I \subseteq [m]$ with $|I| \leq k$ that maximizes $cov(I)$

- Set Cover = Minimization version

(Unweighted) Set Cover

- Given a set system
  - A *universe* U of n elements
  - A collection of m sets $S_1, \dots, S_m$
- Goal: Cover every element using fewest number of sets
- Equivalently, find $I \subseteq [m]$ with $cov(I) = n$ that minimizes $|I|$
- (Max Cover). find $I \subseteq [m]$ with $|I| \leq k$ that maximizes $cov(I)$

- Applications
  - General "covering"-type problem
    - satisfy requirements with fewest resources
  - Place fewest communications tower to cover everyone
  - Place fewest guards to guard everything
  - Pay fewest influencers to influence everyone

(Unweighted) Set Cover

- Given a set system
  - A *universe* U of n elements
  - A collection of m sets $S_1, \ldots, S_m$
- Goal: Cover every element using fewest number of sets
- Equivalently, find $I \subseteq [m]$ with $cov(I) = n$ that minimizes $|I|$
- (Max Cover). find $I \subseteq [m]$ with $|I| \leq k$ that maximizes $cov(I)$

Greedy algorithm

- Initialize $I \leftarrow \emptyset$
- while $cov(I) < n$:
  - Find j^* that maximizes increase in coverage: $cov(I \cup \{j^*\}) - cov(I)$
  - Add j^* to I
- Return I

Greedy Algorithm Analysis

Weighted Set Cover

- Given a set system
  - A *universe* U of n elements
  - A collection of m sets $S_1, \dots, S_m$ with weights $w(S_1), \dots, w(S_m)$
- Goal: Cover every element using sets with minimum total weight
- Equivalently, find $I \subseteq [m]$ with $cov(I) = n$ that minimizes $w(I)$

Greedy Algorithm

- Initialize $I \leftarrow \emptyset$
- while $cov(I) < n$:
  - Find j^* that maximizes marginal density: $\frac{cov(I \cup \{j^*\}) - cov(I)}{w(e_{j^*})}$
  - Add j^* to I
- Return I

Greedy Analysis

Summary

- Marginal Greedy:
  - Good for constraints where size does not matter
  - Matroid constraints
- Density Greedy
  - Good for constraints where size does matter
  - Knapsack-type constraints